

第七章：完全多项式非确定性问题 算法设计与分析

宫秀军

天津大学智能与计算学部

Email: gongxj@tju.edu.cn

April 26, 2025

到现在为止，我们已经学习了五种算法：

- 贪心算法
- 分治算法
- 动态规划算法
- 回溯算法
- 分支限界算法

到现在为止，我们已经学习了五种算法：

- 贪心算法
- 分治算法
- 动态规划算法
- 回溯算法
- 分支限界算法

求解了大致六类问题：

- 排序问题
- 矩阵乘法问题
- 路径规划问题
- 图搜索问题
- 货物装载问题
- 作业调度问题

到现在为止，我们已经学习了五种算法：

- 贪心算法
- 分治算法
- 动态规划算法
- 回溯算法
- 分支限界算法

求解了大致六类问题：

- 排序问题
- 矩阵乘法问题
- 路径规划问题
- 图搜索问题
- 货物装载问题
- 作业调度问题

本章将抽象地研究计算问题

到现在为止，我们已经学习了五种算法：

- 贪心算法
- 分治算法
- 动态规划算法
- 回溯算法
- 分支限界算法

求解了大致六类问题：

- 排序问题
- 矩阵乘法问题
- 路径规划问题
- 图搜索问题
- 货物装载问题
- 作业调度问题

本章将抽象地研究计算问题，即对于给定的问题，我们关注于：

- 是否可以计算？

到现在为止，我们已经学习了五种算法：

- 贪心算法
- 分治算法
- 动态规划算法
- 回溯算法
- 分支限界算法

求解了大致六类问题：

- 排序问题
- 矩阵乘法问题
- 路径规划问题
- 图搜索问题
- 货物装载问题
- 作业调度问题

本章将抽象地研究计算问题，即对于给定的问题，我们关注于：

- 是否可以计算？
- 如果可以计算，如何判断是计算上易解的还是难解的？

到现在为止，我们已经学习了五种算法：

- 贪心算法
- 分治算法
- 动态规划算法
- 回溯算法
- 分支限界算法

求解了大致六类问题：

- 排序问题
- 矩阵乘法问题
- 路径规划问题
- 图搜索问题
- 货物装载问题
- 作业调度问题

本章将抽象地研究计算问题，即对于给定的问题，我们关注于：

- 是否可以计算？
- 如果可以计算，如何判断是计算上易解的还是难解的？
- 如果是难解的，如何判断与哪些已知难解问题是等价的？

到现在为止，我们已经学习了五种算法：

- 贪心算法
- 分治算法
- 动态规划算法
- 回溯算法
- 分支限界算法

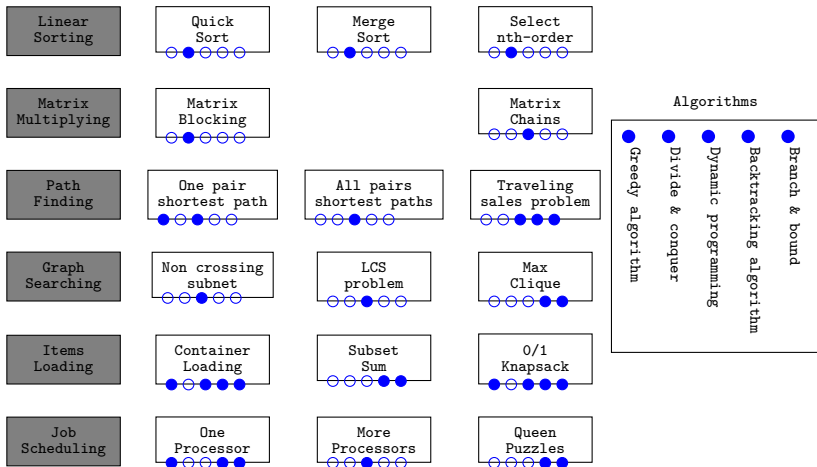
求解了大致六类问题：

- 排序问题
- 矩阵乘法问题
- 路径规划问题
- 图搜索问题
- 货物装载问题
- 作业调度问题

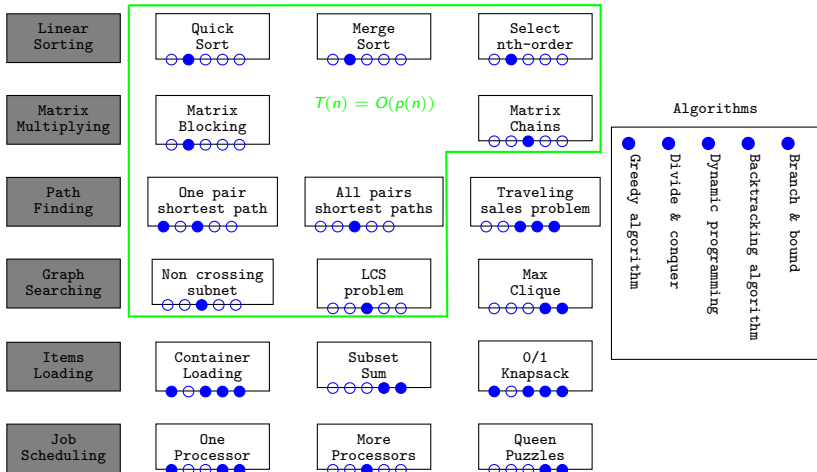
本章将抽象地研究计算问题，即对于给定的问题，我们关注于：

- 是否可以计算？
- 如果可以计算，如何判断是计算上易解的还是难解的？
- 如果是难解的，如何判断与哪些已知难解问题是等价的？
- 如果不可以计算，如何证明？

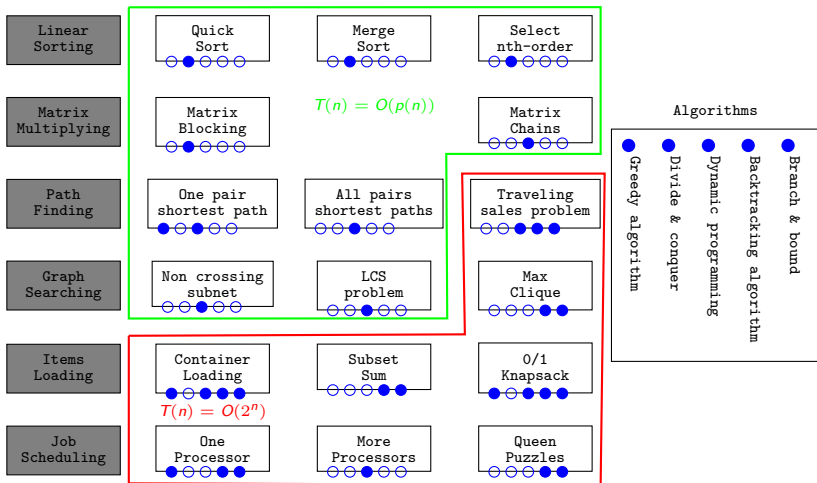
问题与求解算法对应关系矩阵



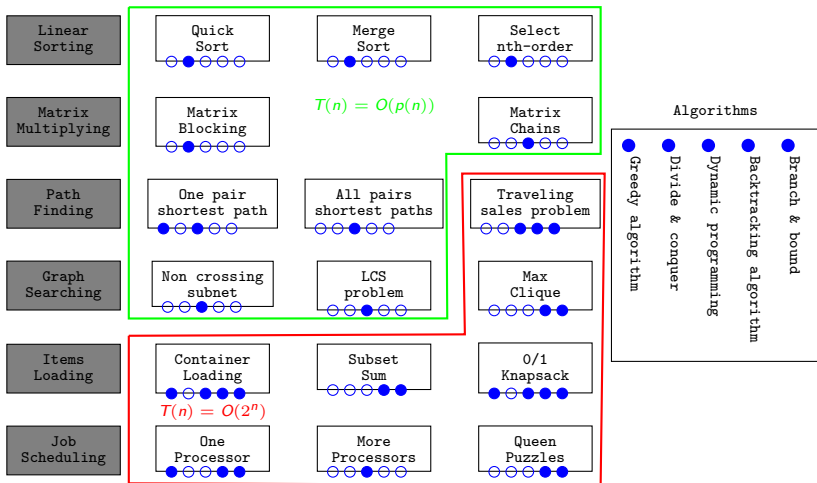
问题与求解算法对应关系矩阵



问题与求解算法对应关系矩阵

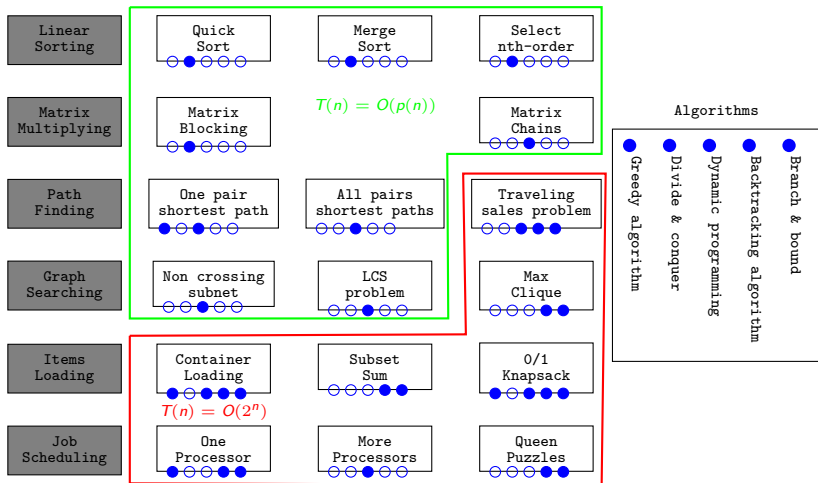


问题与求解算法对应关系矩阵



- 哪些问题“难”？

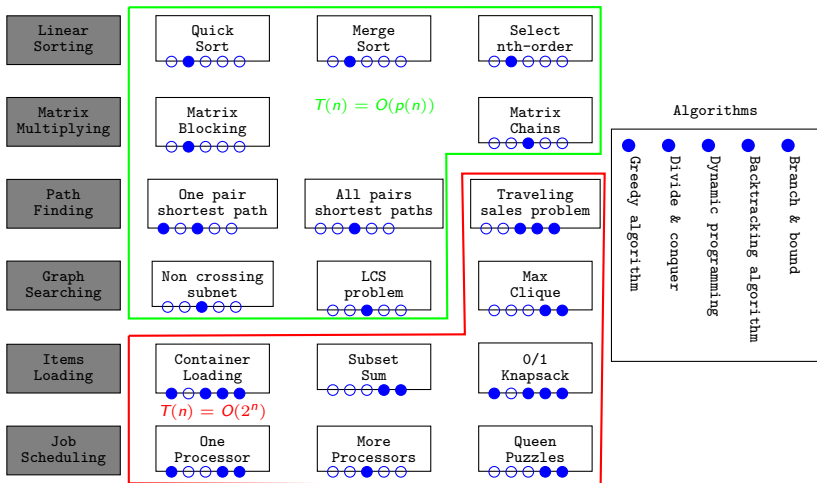
问题与求解算法对应关系矩阵



● 哪些问题“难”？

● 为什么难？

问题与求解算法对应关系矩阵



● 哪些问题“难”？

● 为什么难？

● 如何归类？

通过本章的学习：

通过本章的学习：

① 了解哪些问题是**计算上**易解的、难解的还是不可解的。

⇒ 观察视角

⇒ 判别原则

⇒ 判别方法

通过本章的学习：

① 了解哪些问题是**计算上**易解的、难解的还是不可解的。

- ⇒ 观察视角
- ⇒ 判别原则
- ⇒ 判别方法

② 掌握导致问题**易难**的可能影响因素。

- ⇒ 决策问题与优化问题
- ⇒ 问题实例的输入尺寸
- ⇒ 非确定性算法

通过本章的学习：

- ① 了解哪些问题是**计算上**易解的、难解的还是不可解的。
 - ⇒ 观察视角
 - ⇒ 判别原则
 - ⇒ 判别方法
- ② 掌握导致问题**易难**的可能影响因素。
 - ⇒ 决策问题与优化问题
 - ⇒ 问题实例的输入尺寸
 - ⇒ 非确定性算法
- ③ 理解**NP 问题**的相关定义，能够判别**问题难**的等价性。
 - ⇒ 定义：P, NP, NPC, NP-hard
 - ⇒ 多项式规约
 - ⇒ 难度等价类里面的典型问题

- 1 问题是易还是难?
 - 易解问题
 - 难解问题
 - 不可解问题
- 2 什么是导致问题难解的因素?
- 3 问题难的等价性

我们已经知道很多问题存在**多项式界**的求解算法。例如：

我们已经知道很多问题存在**多项式界**的求解算法。例如：

- 排序问题

我们已经知道很多问题存在**多项式界**的求解算法。例如：

- 排序问题

$$O(n \log n)$$

我们已经知道很多问题存在**多项式界**的求解算法。例如：

- 排序问题
- 查找图中任意两个节点的最短路径问题

$$O(n \log n)$$

我们已经知道很多问题存在**多项式界**的求解算法。例如：

- 排序问题 $O(n \log n)$
- 查找图中任意两个节点的最短路径问题 $O(n^3)$

我们已经知道很多问题存在**多项式界**的求解算法。例如：

- 排序问题 $O(n \log n)$
- 查找图中任意两个节点的最短路径问题 $O(n^3)$
- 计算整数的**完全平方根** (perfect square root) 问题

我们已经知道很多问题存在**多项式界**的求解算法。例如：

- 排序问题 $O(n \log n)$
- 查找图中任意两个节点的最短路径问题 $O(n^3)$
- 计算整数的**完全平方根** (perfect square root) 问题 $O(\log n)$

我们已经知道很多问题存在**多项式界**的求解算法。例如：

- 排序问题 $O(n \log n)$
- 查找图中任意两个节点的最短路径问题 $O(n^3)$
- 计算整数的**完全平方根** (perfect square root) 问题 $O(\log n)$
- ...

我们已经知道很多问题存在**多项式界**的求解算法。例如:

- 排序问题 $O(n \log n)$
- 查找图中任意两个节点的最短路径问题 $O(n^3)$
- 计算整数的**完全平方根** (perfect square root) 问题 $O(\log n)$
- ...

易解问题 [Tractable problems]

如果某问题**存在** 一个多项式界的求解算法，则称该问题为**易解(计算)问题** (computationally tractable)。

我们已经知道很多问题存在**多项式界**的求解算法。例如：

- 排序问题 $O(n \log n)$
- 查找图中任意两个节点的最短路径问题 $O(n^3)$
- 计算整数的**完全平方根** (perfect square root) 问题 $O(\log n)$
- ...

易解问题 [Tractable problems]

如果某问题**存在** 一个多项式界的求解算法，则称该问题为**易解(计算)问题** (computationally tractable)。

- 求解算法的**上界**是**多项式界**，例如 $\log(n)$ ， $n \log(n)$ ， n^2 等；

我们已经知道很多问题存在**多项式界**的求解算法。例如：

- 排序问题 $O(n \log n)$
- 查找图中任意两个节点的最短路径问题 $O(n^3)$
- 计算整数的**完全平方根** (perfect square root) 问题 $O(\log n)$
- ...

易解问题 [Tractable problems]

如果某问题**存在**一个**多项式界**的求解**算法**，则称该问题为**易解(计算)问题** (computationally tractable)。

- 求解算法的**上界**是**多项式界**，例如 $\log(n)$ ， $n \log(n)$ ， n^2 等；
- 这类问题复杂性的上界约束只适用于**合理**的**输入实例尺寸**。

我们已经知道很多问题存在**多项式界**的求解算法。例如：

- 排序问题 $O(n \log n)$
- 查找图中任意两个节点的最短路径问题 $O(n^3)$
- 计算整数的**完全平方根** (perfect square root) 问题 $O(\log n)$
- ...

易解问题 [Tractable problems]

如果某问题**存在** 一个多项式界的求解算法，则称该问题为**易解(计算)问题** (computationally tractable)。

- 求解算法的**上界**是**多项式界**，例如 $\log(n)$ ， $n \log(n)$ ， n^2 等；
- 这类问题复杂性的上界约束只适用于**合理**的**输入实例尺寸**。
- 当前**不是**易解的问题，将来可能会**变为**易解问题。

什么是合理的输入实例尺寸？

什么是合理的输入实例尺寸？

- 在《算法》教材里，查找含有“NP 问题”的段落

什么是合理的输入实例尺寸？

- 在《算法》教材里，查找含有“NP 问题”的段落



什么是合理的输入实例尺寸？

- 在《算法》教材里，查找含有“NP 问题”的段落 ✓
- 在整个互联网里，查找含有“NP 问题”的网页

什么是合理的输入实例尺寸？

- 在《算法》教材里，查找含有“NP 问题”的段落 ✓
- 在整个互联网里，查找含有“NP 问题”的网页 ✗

什么是合理的输入实例尺寸？

- 在《算法》教材里，查找含有“NP 问题”的段落 ✓
- 在整个互联网里，查找含有“NP 问题”的网页 ✗
- 在人类基因组里，查找某个基因

什么是合理的输入实例尺寸？

- 在《算法》教材里，查找含有“NP 问题”的段落 ✓
- 在整个互联网里，查找含有“NP 问题”的网页 ✗
- 在人类基因组里，查找某个基因 ✓

什么是合理的输入实例尺寸？

- 在《算法》教材里，查找含有“NP 问题”的段落 ✓
- 在整个互联网里，查找含有“NP 问题”的网页 ✗
- 在人类基因组里，查找某个基因 ✓
- 在所有物种里，查找某个基因

什么是合理的输入实例尺寸？

- 在《算法》教材里，查找含有“NP 问题”的段落 ✓
- 在整个互联网里，查找含有“NP 问题”的网页 ✗
- 在人类基因组里，查找某个基因 ✓
- 在所有物种里，查找某个基因 ✗

什么是合理的输入实例尺寸？

- 在《算法》教材里，查找含有“NP 问题”的段落 ✓
- 在整个互联网里，查找含有“NP 问题”的网页 ✗
- 在人类基因组里，查找某个基因 ✓
- 在所有物种里，查找某个基因 ✗

期望的输出 与 输入 的关系

难解问题 [Intractable problem]

指的是**不存在**多项式界求解算法的问题。即求解该问题算法的复杂性**至少**是**指数级**的。

难解问题 [Intractable problem]

指的是**不存在**多项式界求解算法的问题。即求解该问题算法的复杂性**至少是指数级的**。

典型的例子包括：

难解问题 [Intractable problem]

指的是**不存在**多项式界求解算法的问题。即求解该问题算法的复杂性**至少是指数级的**。

典型的例子包括：

- 问题本身的**输出**是**输入实例大小** 的超多项式函数

难解问题 [Intractable problem]

指的是**不存在**多项式界求解算法的问题。即求解该问题算法的复杂性**至少是指数级的**。

典型的例子包括：

- 问题本身的**输出**是**输入实例大小** 的超多项式函数
⇒ 组合问题：输出 n 个不同符号的所有组合

难解问题 [Intractable problem]

指的是**不存在**多项式界求解算法的问题。即求解该问题算法的复杂性**至少是指数级的**。

典型的例子包括：

- 问题本身的**输出**是**输入实例大小** 的超多项式函数

⇒ 组合问题：输出 n 个不同符号的所有组合

$O(2^n)$

难解问题 [Intractable problem]

指的是**不存在**多项式界求解算法的问题。即求解该问题算法的复杂性**至少是指数级的**。

典型的例子包括：

- 问题本身的**输出**是**输入实例大小** 的超多项式函数

⇒ 组合问题：输出 n 个不同符号的所有组合

⇒ 排列问题：输出 n 个不同符号的所有排列

$O(2^n)$

难解问题 [Intractable problem]

指的是**不存在**多项式界求解算法的问题。即求解该问题算法的复杂性**至少是指数级的**。

典型的例子包括：

- 问题本身的**输出**是**输入实例大小** 的超多项式函数

⇒ 组合问题：输出 n 个不同符号的所有组合

$O(2^n)$

⇒ 排列问题：输出 n 个不同符号的所有排列

$O(n!)$

难解问题 [Intractable problem]

指的是**不存在**多项式界求解算法的问题。即求解该问题算法的复杂性**至少是指数级的**。

典型的例子包括：

- 问题本身的**输出**是**输入实例大小** 的超多项式函数

⇒ 组合问题：输出 n 个不同符号的所有组合

$O(2^n)$

⇒ 排列问题：输出 n 个不同符号的所有排列

$O(n!)$

⇒ 汉诺塔问题：输出 n 个圆盘的移动步骤

难解问题 [Intractable problem]

指的是**不存在**多项式界求解算法的问题。即求解该问题算法的复杂性**至少是指数级的**。

典型的例子包括：

- 问题本身的**输出**是**输入实例大小** 的超多项式函数

⇒ 组合问题：输出 n 个不同符号的所有组合

$O(2^n)$

⇒ 排列问题：输出 n 个不同符号的所有排列

$O(n!)$

⇒ 汉诺塔问题：输出 n 个圆盘的移动步骤

$(2^n - 1)$

难解问题 [Intractable problem]

指的是**不存在**多项式界求解算法的问题。即求解该问题算法的复杂性**至少是指数级的**。

典型的例子包括：

- 问题本身的**输出**是**输入实例大小** 的超多项式函数
 - ⇒ 组合问题：输出 n 个不同符号的所有组合 $O(2^n)$
 - ⇒ 排列问题：输出 n 个不同符号的所有排列 $O(n!)$
 - ⇒ 汉诺塔问题：输出 n 个圆盘的移动步骤 $(2^n - 1)$
- 有些问题，尽管**不需要**超出多项式界的输出，但仍然**无法**找到求解的多项式界算法

难解问题 [Intractable problem]

指的是**不存在**多项式界求解算法的问题。即求解该问题算法的复杂性**至少是指数级的**。

典型的例子包括：

- 问题本身的**输出**是**输入实例大小** 的超多项式函数
 - ⇒ 组合问题：输出 n 个不同符号的所有组合 $O(2^n)$
 - ⇒ 排列问题：输出 n 个不同符号的所有排列 $O(n!)$
 - ⇒ 汉诺塔问题：输出 n 个圆盘的移动步骤 $(2^n - 1)$
- 有些问题，尽管**不需要**超出多项式界的输出，但仍然**无法**找到求解的多项式界算法
 - ⇒ 0/1 背包问题
 - ⇒ 旅行商问题 (TSP)
 - ⇒ 查找完全子图问题

素数检验问题: 给定任意正整数 n , 检验 n 是否存在大于 1 的整数因子

素数检验问题: 给定任意正整数 n , 检验 n 是否存在大于 1 的整数因子

穷举法:

```
1 def isPrime(n):
2     found = False
3     for i in range(2, n):
4         if i % n == 0:
5             found = True
6             break
7     return found
```

素数检验问题: 给定任意正整数 n , 检验 n 是否存在大于 1 的整数因子

穷举法:

```
1 def isPrime(n):  
2     found = False  
3     for i in range(2, n):  
4         if i % n == 0:  
5             found = True  
6             break  
7     return found
```

该算法具有伪多项式界的复杂性（下一节证明）

素数检验问题: 给定任意正整数 n , 检验 n 是否存在大于 1 的整数因子

穷举法:

```
1 def isPrime(n):
2     found = False
3     for i in range(2, n):
4         if i % n == 0:
5             found = True
6             break
7     return found
```

该算法具有伪多项式界的复杂性 (下一节证明)

直到 2004 年, AKS(M. Agrawal, N. Kayal, and N. Saxena) 在美国数学年报上提出了复杂性是 n 的位数的多项式函数的算法。

素数检验问题: 给定任意正整数 n , 检验 n 是否存在大于 1 的整数因子

穷举法:

```
1 def isPrime(n):
2     found = False
3     for i in range(2, n):
4         if i % n == 0:
5             found = True
6             break
7     return found
```

该算法具有伪多项式界的复杂性 (下一节证明)

直到 2004 年, AKS(M. Agrawal, N. Kayal, and N. Saxena) 在美国数学年报上提出了复杂性是 n 的位数的多项式函数的算法。

- 算法扩展了费马 (Fermat) 小定理: $a^n \equiv a \pmod{n}$
- 三位作者因此获得了 2006 年度哥德尔(Gödel) 奖和富尔克森(Fulkerson) 奖

不可解问题 [unsolvable problem]

指的是不能被确定型图灵机求解的计算问题，即不存在任何算法能够求解的计算问题。

不可解问题 [unsolvable problem]

指的是不能被确定型图灵机求解的计算问题，即不存在任何算法能够求解的计算问题。

停机问题 (HALTING PROBLEM): 判断任意一个程序 是否能在有限的时间 之内结束运行。可以描述为:

不可解问题 [unsolvable problem]

指的是不能被确定型图灵机求解的计算问题，即不存在任何算法能够求解的计算问题。

停机问题 (HALTING PROBLEM): 判断任意一个程序 是否能在有限的时间 之内结束运行。可以描述为:

- 判定问题: 是否存在一个程序 P , 对于任意输入的程序 w , 能够判断 w 会在有限时间内结束或者死循环。

不可解问题 [unsolvable problem]

指的是不能被确定型图灵机求解的计算问题，即不存在任何算法能够求解的计算问题。

停机问题 (HALTING PROBLEM): 判断任意一个程序 是否能在有限的时间 之内结束运行。可以描述为:

- 判定问题: 是否存在一个程序 P , 对于任意输入的程序 w , 能够判断 w 会在有限时间内结束或者死循环。
- 形式语言: 给定一个图灵机 T 和一个任意语言集合 S , T 是否会最终停机于每一个 $s \in S$ 。

不可解问题 [unsolvable problem]

指的是不能被确定型图灵机求解的计算问题，即不存在任何算法能够求解的计算问题。

停机问题 (HALTING PROBLEM): 判断任意一个程序 是否能在有限的时间 之内结束运行。可以描述为:

- 判定问题: 是否存在一个程序 P , 对于任意输入的程序 w , 能够判断 w 会在有限时间内结束或者死循环。
- 形式语言: 给定一个图灵机 T 和一个任意语言集合 S , T 是否会最终停机于每一个 $s \in S$ 。

艾伦·图灵在 1936 年用对角论证法 证明了不存在解决停机问题的通用算法。

停机问题的证明

假设停机问题有解，则存在算法 $\text{halt}(P, I)$ 可以判断对于输入 I 时程序 P 是否停机，即：

$$\text{halt}(P, I) = \begin{cases} \text{Yes} & \text{if 输入 } I, P \text{ 停机} \\ \text{No} & \text{if 输入 } I, P \text{ 进入死循环} \end{cases}$$

停机问题的证明

假设停机问题有解, 则存在算法 $\text{halt}(P, I)$ 可以判断对于输入 I 时程序 P 是否停机, 即:

$$\text{halt}(P, I) = \begin{cases} \text{Yes} & \text{if 输入 } I, P \text{ 停机} \\ \text{No} & \text{if 输入 } I, P \text{ 进入死循环} \end{cases}$$

我们构造下面的程序 $Z(P)$:

```
1 Z(P){  
2   a: if halt(P,P) then  
3       goto a;  
4   else halt;  
5 }
```

停机问题的证明

假设停机问题有解, 则存在算法 $\text{halt}(P, I)$ 可以判断对于输入 I 时程序 P 是否停机, 即:

$$\text{halt}(P, I) = \begin{cases} \text{Yes} & \text{if 输入 } I, P \text{ 停机} \\ \text{No} & \text{if 输入 } I, P \text{ 进入死循环} \end{cases}$$

我们构造下面的程序 $Z(P)$:

```
1 Z(P){  
2   a: if halt(P,P) then  
3       goto a;  
4   else halt;  
5 }
```

功能: 当程序 P 读入其本身 P 时, 如果 halt 算法判定其停机, 则 Z 进入死循环, 否则 Z 停机。

停机问题的证明

假设停机问题有解, 则存在算法 $\text{halt}(P, I)$ 可以判断对于输入 I 时程序 P 是否停机, 即:

$$\text{halt}(P, I) = \begin{cases} \text{Yes} & \text{if 输入 } I, P \text{ 停机} \\ \text{No} & \text{if 输入 } I, P \text{ 进入死循环} \end{cases}$$

我们构造下面的程序 $Z(P)$:

```
1 Z(P){  
2   a: if halt(P,P) then  
3       goto a;  
4   else halt;  
5 }
```

功能: 当程序 P 读入其本身 P 时, 如果 halt 算法判定其停机, 则 Z 进入死循环, 否则 Z 停机。

$Z(Z)$ 是否停机?

停机问题的证明, cont.

Case 1: 程序 Z 关于输入 Z 停机: halt 判定其停机, 而程序 Z 判定其进入死循环

停机问题的证明, cont.

Case 1: 程序 Z 关于输入 Z 停机: halt 判定其停机, 而程序 Z 判定其进入死循环 **矛盾**

停机问题的证明, cont.

- Case 1: 程序 Z 关于输入 Z 停机: halt 判定其停机, 而程序 Z 判定其进入死循环 **矛盾**
- Case 2: 程序 Z 关于输入 Z 进入死循环: halt 判定其进入死循环, 而程序 Z 判定其停机

停机问题的证明, cont.

- Case 1: 程序 Z 关于输入 Z 停机: halt 判定其停机, 而程序 Z 判定其进入死循环 矛盾
- Case 2: 程序 Z 关于输入 Z 进入死循环: halt 判定其进入死循环, 而程序 Z 判定其停机 矛盾

停机问题的证明, cont.

- Case 1: 程序 Z 关于输入 Z 停机: halt 判定其停机, 而程序 Z 判定其进入死循环 矛盾
- Case 2: 程序 Z 关于输入 Z 进入死循环: halt 判定其进入死循环, 而程序 Z 判定其停机 矛盾

因而 $\text{halt}(P, I)$ 不存在, 停机问题是不可解 (计算) 的!

停机问题的证明, cont.

- Case 1: 程序 Z 关于输入 Z 停机: halt 判定其停机, 而程序 Z 判定其进入死循环 矛盾
- Case 2: 程序 Z 关于输入 Z 进入死循环: halt 判定其进入死循环, 而程序 Z 判定其停机 矛盾

因而 $\text{halt}(P, I)$ 不存在, 停机问题是不可解 (计算) 的!

类似的不可计算问题还包括:

- 程序等价性问题 (Program Equivalence)
- 断言验证问题 (Statement Verification)
- 定理识别问题 (Theory Recognition)

小结

了解哪些问题是**计算上**易解的、难解的还是不可解的。

- 观察视角
- 判别原则
- 判别方法

小结

了解哪些问题是**计算上**易解的、难解的还是不可解的。

- 观察视角 $\Rightarrow$ **计算复杂性**
- 判别原则
- 判别方法

小结

了解哪些问题是**计算上**易解的、难解的还是不可解的。

- 观察视角 $\Rightarrow$ **计算复杂性**
- 判别原则 $\Rightarrow$ **难易：多项式界；不可解：可计算性**
- 判别方法

小结

了解哪些问题是**计算上**易解的、难解的还是不可解的。

- 观察视角 $\Rightarrow$ **计算复杂性**
- 判别原则 $\Rightarrow$ **难易：多项式界；不可解：可计算性**
- 判别方法 $\Rightarrow$ **存在性：举例；不存在：反证法**

- ① 问题是易还是难?
- ② 什么是导致问题难解的因素?
 - 决策问题与优化问题
 - 输入实例的长度
 - 非确定性算法
- ③ 问题难的等价性

抽象问题: 抽象问题 Q 定义为问题实例空间 I 和问题解空间 S 上的二元关系。

抽象问题: 抽象问题 Q 定义为问题实例空间 I 和问题解空间 S 上的二元关系。

Ex1: 判断图中两点之间是否存在长度不超过 k 的路径

抽象问题: 抽象问题 Q 定义为问题实例空间 I 和问题解空间 S 上的二元关系。

Ex1: 判断图中两点之间是否存在长度不超过 k 的路径

- 问题实例空间: $I = \{(G, u, v, k) \mid u, v \in G, k \geq 0 \in R\}$

抽象问题: 抽象问题 Q 定义为问题实例空间 I 和问题解空间 S 上的二元关系。

Ex1: 判断图中两点之间是否存在长度不超过 k 的路径

- 问题实例空间: $I = \{(G, u, v, k) \mid u, v \in G, k \geq 0 \in R\}$
- 问题的解空间: $S = \{yes, no\}$

抽象问题: 抽象问题 Q 定义为问题实例空间 I 和问题解空间 S 上的二元关系。

Ex1: 判断图中两点之间是否存在长度不超过 k 的路径

- 问题实例空间: $I = \{(G, u, v, k) | u, v \in G, k \geq 0 \in R\}$
- 问题的解空间: $S = \{yes, no\}$
- 问题解:

$$SP = \begin{cases} yes & \text{if 存在一条路径 } s, \text{ 使得 } path(s) \leq k \\ no & \text{otherwise} \end{cases}$$

抽象问题: 抽象问题 Q 定义为问题实例空间 I 和问题解空间 S 上的二元关系。

Ex1: 判断图中两点之间是否存在长度不超过 k 的路径

- 问题实例空间: $I = \{(G, u, v, k) | u, v \in G, k \geq 0 \in R\}$
- 问题的解空间: $S = \{yes, no\}$
- 问题解:

$$SP = \begin{cases} yes & \text{if 存在一条路径 } s, \text{ 使得 } path(s) \leq k \\ no & \text{otherwise} \end{cases}$$

Ex2: 找出图中两点之间的最短路径

抽象问题: 抽象问题 Q 定义为问题实例空间 I 和问题解空间 S 上的二元关系。

Ex1: 判断图中两点之间是否存在长度不超过 k 的路径

- 问题实例空间: $I = \{(G, u, v, k) | u, v \in G, k \geq 0 \in R\}$
- 问题的解空间: $S = \{yes, no\}$
- 问题解:

$$SP = \begin{cases} yes & \text{if 存在一条路径 } s, \text{ 使得 } path(s) \leq k \\ no & \text{otherwise} \end{cases}$$

Ex2: 找出图中两点之间的最短路径

- 问题实例空间: $I = \{(G, u, v) | u, v \in G\}$

抽象问题: 抽象问题 Q 定义为问题实例空间 I 和问题解空间 S 上的二元关系。

Ex1: 判断图中两点之间是否存在长度不超过 k 的路径

- 问题实例空间: $I = \{(G, u, v, k) | u, v \in G, k \geq 0 \in R\}$
- 问题的解空间: $S = \{yes, no\}$
- 问题解:

$$SP = \begin{cases} yes & \text{if 存在一条路径 } s, \text{ 使得 } path(s) \leq k \\ no & \text{otherwise} \end{cases}$$

Ex2: 找出图中两点之间的最短路径

- 问题实例空间: $I = \{(G, u, v) | u, v \in G\}$
- 问题的解空间: $S = \{(uc_1 \cdots c_pv) | p \geq 0, \}$

抽象问题: 抽象问题 Q 定义为问题实例空间 I 和问题解空间 S 上的二元关系。

Ex1: 判断图中两点之间是否存在长度不超过 k 的路径

- 问题实例空间: $I = \{(G, u, v, k) | u, v \in G, k \geq 0 \in R\}$
- 问题的解空间: $S = \{yes, no\}$
- 问题解:

$$SP = \begin{cases} yes & \text{if 存在一条路径 } s, \text{ 使得 } path(s) \leq k \\ no & \text{otherwise} \end{cases}$$

Ex2: 找出图中两点之间的最短路径

- 问题实例空间: $I = \{(G, u, v) | u, v \in G\}$
- 问题的解空间: $S = \{(uc_1 \cdots c_pv) | p \geq 0, \}$
- 最优解: $SP = \{s | s \in S \text{ and } path(s) \leq path(s') \text{ for } \forall s' \in S\}$

抽象问题: 抽象问题 Q 定义为问题实例空间 I 和问题解空间 S 上的二元关系。

Ex1: 判断图中两点之间是否存在长度不超过 k 的路径

- 问题实例空间: $I = \{(G, u, v, k) | u, v \in G, k \geq 0 \in R\}$
- 问题的解空间: $S = \{yes, no\}$
- 问题解:

$$SP = \begin{cases} yes & \text{if 存在一条路径 } s, \text{ 使得 } path(s) \leq k \\ no & \text{otherwise} \end{cases}$$

Ex2: 找出图中两点之间的最短路径

- 问题实例空间: $I = \{(G, u, v) | u, v \in G\}$
- 问题的解空间: $S = \{(uc_1 \cdots c_pv) | p \geq 0, \}$
- 最优解: $SP = \{s | s \in S \text{ and } path(s) \leq path(s') \text{ for } \forall s' \in S\}$
- 最优值: $SV = k$, 满足:
 - $\Rightarrow SP(G, u, v, k) = yes$
 - $\Rightarrow \text{if } \exists k' \text{ s.t. } SP(G, u, v, k') = yes \text{ then } k \leq k'$

- **决策问题**(Decision problem) 是只要求回答'yes' 或'no' 的问题，它的解空间只含有**两个元素** $\{yes, no\}$ ，有时也称为**判定问题**。如 Ex1

- **决策问题**(Decision problem) 是只要求回答'yes' 或'no' 的问题，它的解空间只含有**两个元素** $\{yes, no\}$ ，有时也称为**判定问题**。如 Ex1
- **优化问题**(Optimization problem) 要求在所有的可行解里找出使得目标函数**最大**或**最小**的解。如 Ex2

- **决策问题**(Decision problem) 是只要求回答'yes' 或'no' 的问题，它的解空间只含有**两个元素** $\{yes, no\}$ ，有时也称为**判定问题**。如 Ex1
- **优化问题**(Optimization problem) 要求在所有的可行解里找出使得目标函数**最大**或**最小**的解。如 Ex2
- 任何**优化问题**，都可以转化为相应的**决策问题**。

- **决策问题**(Decision problem) 是只要求回答'yes' 或'no' 的问题，它的解空间只含有**两个元素** $\{yes, no\}$ ，有时也称为**判定问题**。如 Ex1
- **优化问题**(Optimization problem) 要求在所有的可行解里找出使得目标函数**最大**或**最小**的解。如 Ex2
- 任何**优化问题**，都可以转化为相应的**决策问题**。
⇒ **0/1 背包问题**：给定效益值 b ，是否存在一种装箱方案 s ，使得 $p(s) > b$?

- **决策问题**(Decision problem) 是只要求回答'yes' 或'no' 的问题，它的解空间只含有**两个元素** $\{yes, no\}$ ，有时也称为**判定问题**。如 Ex1
- **优化问题**(Optimization problem) 要求在所有的可行解里找出使得目标函数**最大**或**最小**的解。如 Ex2
- 任何**优化问题**，都可以转化为相应的**决策问题**。
 - ⇒ **0/1 背包问题**：给定效益值 b ，是否存在一种装箱方案 s ，使得 $p(s) > b$?
 - ⇒ **TSP 问题**：给定数值 b ，是否存在一个哈密顿回路 l ，使得 $path(l) \leq b$?

- **决策问题**(Decision problem) 是只要求回答'yes' 或'no' 的问题，它的解空间只含有**两个元素** $\{yes, no\}$ ，有时也称为**判定问题**。如 Ex1
- **优化问题**(Optimization problem) 要求在所有的可行解里找出使得目标函数**最大**或**最小**的解。如 Ex2
- 任何**优化问题**，都可以转化为相应的**决策问题**。
 - ⇒ **0/1 背包问题**：给定效益值 b ，是否存在一种装箱方案 s ，使得 $p(s) > b$?
 - ⇒ **TSP 问题**：给定数值 b ，是否存在一个哈密顿回路 l ，使得 $path(l) \leq b$?
 - ⇒ 如果求解优化问题的复杂性是多项式界，那么求解相应决策问题的复杂性**也一定是多项式界**。

- **决策问题**(Decision problem) 是只要求回答'yes' 或'no' 的问题, 它的解空间只含有**两个元素** $\{yes, no\}$, 有时也称为**判定问题**。如 Ex1
- **优化问题**(Optimization problem) 要求在所有的可行解里找出使得目标函数**最大**或**最小**的解。如 Ex2
- 任何**优化问题**, 都可以转化为相应的**决策问题**。
 - ⇒ **0/1 背包问题** : 给定效益值 b , 是否存在一种装箱方案 s , 使得 $p(s) > b$?
 - ⇒ **TSP 问题** : 给定数值 b , 是否存在一个哈密顿回路 l , 使得 $path(l) \leq b$?
 - ⇒ 如果求解优化问题的复杂性是多项式界, 那么求解相应决策问题的复杂性**也一定是多项式界**。
 - ⇒ 反之, 如果求解相应决策问题的复杂性是多项式界, 对于最大/最小问题, 可以通过**二分查找**的方式来求解其优化问题, 复杂性也是多项式界的。

- **决策问题**(Decision problem) 是只要求回答'yes' 或'no' 的问题，它的解空间只含有**两个元素** $\{yes, no\}$ ，有时也称为**判定问题**。如 Ex1
- **优化问题**(Optimization problem) 要求在所有的可行解里找出使得目标函数**最大**或**最小**的解。如 Ex2
- 任何**优化问题**，都可以转化为相应的**决策问题**。
 - ⇒ **0/1 背包问题**：给定效益值 b ，是否存在一种装箱方案 s ，使得 $p(s) > b$?
 - ⇒ **TSP 问题**：给定数值 b ，是否存在一个哈密顿回路 l ，使得 $path(l) \leq b$?
 - ⇒ 如果求解优化问题的复杂性是多项式界，那么求解相应决策问题的复杂性**也一定是多项式界**。
 - ⇒ 反之，如果求解相应决策问题的复杂性是多项式界，对于最大/最小问题，可以通过**二分查找**的方式来求解其优化问题，复杂性也是多项式界的。
 - ⇒ 从这个意义上讲，二者之间的复杂性是**近似等价**的，故在难度等价性问题中，我们只讨论**决策问题**的情况。

算法的**输入长度** 定义为**输入实例** 的**二进制编码长度**。

算法的**输入长度** 定义为**输入实例** 的**二进制编码长度**。

实例 1: 0/1 背包问题:

算法的**输入长度** 定义为**输入实例** 的**二进制编码长度**。

实例 1: 0/1 背包问题:

- 输入实例: $n, c, w = (w_1, \dots, w_n), p = (p_1, \dots, p_n)$

算法的**输入长度** 定义为**输入实例** 的**二进制编码长度**。

实例 1: 0/1 背包问题:

- 输入实例: $n, c, w = (w_1, \dots, w_n), p = (p_1, \dots, p_n)$
- 实例长度: $m = \log_2 n + \log_2 c + \sum \log_2 p_i + \sum \log_2 w_i$

算法的**输入长度** 定义为**输入实例** 的**二进制编码长度**。

实例 1: 0/1 背包问题:

- 输入实例: $n, c, w = (w_1, \dots, w_n), p = (p_1, \dots, p_n)$
- 实例长度: $m = \log_2 n + \log_2 c + \sum \log_2 p_i + \sum \log_2 w_i$
- 算法复杂性: $T = \Theta(nc)$ 是 **伪**多项式

算法的**输入长度** 定义为**输入实例** 的**二进制编码长度**。

实例 1: 0/1 背包问题:

- 输入实例: $n, c, w = (w_1, \dots, w_n), p = (p_1, \dots, p_n)$
- 实例长度: $m = \log_2 n + \log_2 c + \sum \log_2 p_i + \sum \log_2 w_i$
- 算法复杂性: $T = \Theta(nc)$ 是 **伪**多项式

Proof.

假设 $p_i = O(c), w_i = O(c),$



算法的**输入长度** 定义为**输入实例** 的**二进制编码长度**。

实例 1: 0/1 背包问题:

- 输入实例: $n, c, w = (w_1, \dots, w_n), p = (p_1, \dots, p_n)$
- 实例长度: $m = \log_2 n + \log_2 c + \sum \log_2 p_i + \sum \log_2 w_i$
- 算法复杂性: $T = \Theta(nc)$ 是 **伪**多项式

Proof.

假设 $p_i = O(c), w_i = O(c),$

$$m = \log_2 n + \log_2 c + \sum \log_2 p_i + \sum \log_2 w_i$$



算法的**输入长度** 定义为**输入实例** 的**二进制编码长度**。

实例 1: 0/1 背包问题:

- 输入实例: $n, c, w = (w_1, \dots, w_n), p = (p_1, \dots, p_n)$
- 实例长度: $m = \log_2 n + \log_2 c + \sum \log_2 p_i + \sum \log_2 w_i$
- 算法复杂性: $T = \Theta(nc)$ 是 **伪**多项式

Proof.

假设 $p_i = O(c), w_i = O(c),$

$$\begin{aligned} m &= \log_2 n + \log_2 c + \sum \log_2 p_i + \sum \log_2 w_i \\ &< \log_2 n + \log_2 c + n * \log_2 c + n * \log_2 c \end{aligned}$$



算法的**输入长度** 定义为**输入实例** 的**二进制编码长度**。

实例 1: 0/1 背包问题:

- 输入实例: $n, c, w = (w_1, \dots, w_n), p = (p_1, \dots, p_n)$
- 实例长度: $m = \log_2 n + \log_2 c + \sum \log_2 p_i + \sum \log_2 w_i$
- 算法复杂性: $T = \Theta(nc)$ 是 **伪**多项式

Proof.

假设 $p_i = O(c), w_i = O(c)$,

$$\begin{aligned} m &= \log_2 n + \log_2 c + \sum \log_2 p_i + \sum \log_2 w_i \\ &< \log_2 n + \log_2 c + n * \log_2 c + n * \log_2 c \\ &= \log_2 n + \log_2 c + 2 * n * \log_2 c \end{aligned}$$



算法的**输入长度** 定义为**输入实例** 的**二进制编码长度**。

实例 1: 0/1 背包问题:

- 输入实例: $n, c, w = (w_1, \dots, w_n), p = (p_1, \dots, p_n)$
- 实例长度: $m = \log_2 n + \log_2 c + \sum \log_2 p_i + \sum \log_2 w_i$
- 算法复杂性: $T = \Theta(nc)$ 是 **伪**多项式

Proof.

假设 $p_i = O(c), w_i = O(c),$

$$\begin{aligned} m &= \log_2 n + \log_2 c + \sum \log_2 p_i + \sum \log_2 w_i \\ &< \log_2 n + \log_2 c + n * \log_2 c + n * \log_2 c \\ &= \log_2 n + \log_2 c + 2 * n * \log_2 c \\ m &= \Theta(n \log_2 c) \end{aligned}$$



算法的**输入长度** 定义为**输入实例** 的**二进制编码长度**。

实例 1: 0/1 背包问题:

- 输入实例: $n, c, w = (w_1, \dots, w_n), p = (p_1, \dots, p_n)$
- 实例长度: $m = \log_2 n + \log_2 c + \sum \log_2 p_i + \sum \log_2 w_i$
- 算法复杂性: $T = \Theta(nc)$ 是 **伪**多项式

Proof.

假设 $p_i = O(c), w_i = O(c)$,

$$\begin{aligned} m &= \log_2 n + \log_2 c + \sum \log_2 p_i + \sum \log_2 w_i \\ &< \log_2 n + \log_2 c + n * \log_2 c + n * \log_2 c \\ &= \log_2 n + \log_2 c + 2 * n * \log_2 c \\ m &= \Theta(n \log_2 c) \end{aligned}$$

所以 $T = \Theta(nc) = \Theta(n2^m)$



实例 2. 合数问题: 对于任意正整数 n , 是否存在两个大于 1 的正整数 k 和 p , 使得 $n = kp$?

实例 2. 合数问题: 对于任意正整数 n , 是否存在两个大于 1 的正整数 k 和 p , 使得 $n = kp$?

```
1  int ComNum(int n)
2  {
3      factor=0;
4      for (j=2;j<n;j++)
5          if ((n mod j)==0)
6              factor=j;
7              break;
8      return factor
9  }
```

实例 2. 合数问题: 对于任意正整数 n , 是否存在两个大于 1 的正整数 k 和 p , 使得 $n = kp$?

```
1  int ComNum(int n)
2  {
3      factor=0;
4      for (j=2;j<n;j++)
5          if ((n mod j)==0)
6              factor=j;
7              break;
8      return factor
9  }
```

算法的时间复杂性: $T(n) = \Theta(n \log^2 n)$ 是 伪多项式。

实例 2. 合数问题: 对于任意正整数 n , 是否存在两个大于 1 的正整数 k 和 p , 使得 $n = kp$?

```
1  int ComNum(int n)
2  {
3      factor=0;
4      for (j=2;j<n;j++)
5          if ((n mod j)==0)
6              factor=j;
7              break;
8      return factor
9  }
```

算法的时间复杂性: $T(n) = \Theta(n \log^2 n)$ 是 **伪**多项式。

Proof.

实例 2. 合数问题: 对于任意正整数 n , 是否存在两个大于 1 的正整数 k 和 p , 使得 $n = kp$?

```
1  int ComNum(int n)
2  {
3      factor=0;
4      for (j=2;j<n;j++)
5          if ((n mod j)==0)
6              factor=j;
7              break;
8      return factor
9  }
```

算法的时间复杂性: $T(n) = \Theta(n \log^2 n)$ 是 **伪**多项式。

Proof.

输入实例: n , 实例长度: $m = \log_2 n$

实例 2. 合数问题: 对于任意正整数 n , 是否存在两个大于 1 的正整数 k 和 p , 使得 $n = kp$?

```
1  int ComNum(int n)
2  {
3      factor=0;
4      for (j=2;j<n;j++)
5          if ((n mod j)==0)
6              factor=j;
7              break;
8      return factor
9  }
```

算法的时间复杂性: $T(n) = \Theta(n \log^2 n)$ 是 **伪**多项式。

Proof.

输入实例: n , 实例长度: $m = \log_2 n$

$$T(n) = \Theta(n \log^2 n)$$

实例 2. 合数问题: 对于任意正整数 n , 是否存在两个大于 1 的正整数 k 和 p , 使得 $n = kp$?

```

1  int ComNum(int n)
2  {
3      factor=0;
4      for (j=2;j<n;j++)
5          if ((n mod j)==0)
6              factor=j;
7              break;
8      return factor
9  }
```

算法的时间复杂性: $T(n) = \Theta(n \log^2 n)$ 是 **伪**多项式。

Proof.

输入实例: n , 实例长度: $m = \log_2 n$

$$T(n) = \Theta(n \log^2 n)$$

$$\Theta(m^3 \cdot 2^m)$$

实例 2. 合数问题: 对于任意正整数 n , 是否存在两个大于 1 的正整数 k 和 p , 使得 $n = kp$?

```

1  int ComNum(int n)
2  {
3      factor=0;
4      for (j=2;j<n;j++)
5          if ((n mod j)==0)
6              factor=j;
7              break;
8      return factor
9  }
```

算法的时间复杂性: $T(n) = \Theta(n \log^2 n)$ 是 **伪**多项式。

Proof.

输入实例: n , 实例长度: $m = \log_2 n$

$$T(n) = \Theta(n \log^2 n)$$

$$\Theta(m \cdot 2^m \cdot m)$$

实例 2. 合数问题: 对于任意正整数 n , 是否存在两个大于 1 的正整数 k 和 p , 使得 $n = kp$?

```

1  int ComNum(int n)
2  {
3      factor=0;
4      for (j=2;j<n;j++)
5          if ((n mod j)==0)
6              factor=j;
7              break;
8      return factor
9  }
```

算法的时间复杂性: $T(n) = \Theta(n \log^2 n)$ 是 **伪**多项式。

Proof.

输入实例: n , 实例长度: $m = \log_2 n$

$$T(n) = \Theta(n \log^2 n)$$

$$\Theta(m \cdot 2^m \cdot m^2)$$

实例 3: 快速排序算法的平均时间复杂性是 $T(n) = \Theta(n \log n)$

实例 3: 快速排序算法的平均时间复杂度是 $T(n) = \Theta(n \log n)$

该算法的复杂度是否为伪多项式？

实例 3: 快速排序算法的平均时间复杂性是 $T(n) = \Theta(n \log n)$

该算法的复杂性是否为伪多项式？

输入实例: $n, a_1, \dots, a_n$, 实例长度: $m = \log n + \sum_{i=1}^n \log a_i$

实例 3: 快速排序算法的平均时间复杂性是 $T(n) = \Theta(n \log n)$

该算法的复杂性是否为伪多项式？

输入实例: $n, a_1, \dots, a_n$, 实例长度: $m = \log n + \sum_{i=1}^n \log a_i$

$$\text{令 } c = \max_{i=1}^n a_i$$

实例 3: 快速排序算法的平均时间复杂度是 $T(n) = \Theta(n \log n)$

该算法的复杂度是否为**伪多项式**?

输入实例: $n, a_1, \dots, a_n$, 实例长度: $m = \log n + \sum_{i=1}^n \log a_i$

$$\text{令 } c = \max_{i=1}^n a_i$$

$$\begin{aligned} m &< \log n + n * \log c \\ &= \Theta(n) \end{aligned}$$

实例 3: 快速排序算法的平均时间复杂性是 $T(n) = \Theta(n \log n)$

该算法的复杂性是否为**伪多项式**?

输入实例: $n, a_1, \dots, a_n$, 实例长度: $m = \log n + \sum_{i=1}^n \log a_i$

$$\text{令 } c = \max_{i=1}^n a_i$$

$$\begin{aligned} m &< \log n + n * \log c \\ &= \Theta(n) \end{aligned}$$

$$\begin{aligned} T(n) &= \Theta(n \log n) \\ &= \Theta(m \log m) \end{aligned}$$

实例 3: 快速排序算法的平均时间复杂性是 $T(n) = \Theta(n \log n)$

该算法的复杂性是否为伪多项式？

输入实例: $n, a_1, \dots, a_n$, 实例长度: $m = \log n + \sum_{i=1}^n \log a_i$

$$\text{令 } c = \max_{i=1}^n a_i$$

$$m < \log n + n * \log c$$

$$= \Theta(n)$$

$$T(n) = \Theta(n \log n)$$

$$= \Theta(m \log m)$$

注意：算法复杂性研究的是输入实例长度与复杂性之间的关系，而非输入特征。

实例 3: 快速排序算法的平均时间复杂性是 $T(n) = \Theta(n \log n)$

该算法的复杂性是否为**伪多项式**？

输入实例: $n, a_1, \dots, a_n$, 实例长度: $m = \log n + \sum_{i=1}^n \log a_i$

$$\text{令 } c = \max_{i=1}^n a_i$$

$$m < \log n + n * \log c$$

$$= \Theta(n)$$

$$T(n) = \Theta(n \log n)$$

$$= \Theta(m \log m)$$

注意：算法复杂性研究的是**输入实例长度**与**复杂性**之间的关系，**而非输入特征**。当我们用近似符号表示算法复杂性时，需要判断输入特征与输入长度是否是**多项式**关系：

- 是：可以使用近似符号表示输入特征与复杂性的关系
- 否：不能直接使用，需要推导输入长度与复杂性的关系

非确定性算法指的是算法**多次**运行**同样**的输入实例时，可能表现出**不同的行为**（例如某个时刻有多个动作可供选择），从而导致不同的**运行结果**。

非确定性算法指的是算法**多次**运行**同样**的输入实例时，可能表现出**不同的行为**（例如某个时刻有多个动作可供选择），从而导致不同的**运行结果**。

导致不确定的原因：

- 概率算法：动作（指令）的执行依赖于随机数的产生。
- 并发算法：动作（指令）的执行结果依赖于竞态条件（race condition）。

非确定性算法指的是算法**多次**运行**同样**的输入实例时，可能表现出**不同的行为**（例如某个时刻有多个动作可供选择），从而导致不同的**运行结果**。

导致不确定的原因：

- 概率算法：动作（指令）的执行依赖于随机数的产生。
- 并发算法：动作（指令）的执行结果依赖于竞态条件（race condition）。

```
1 Void nondetA(String w)
2     String c=genCertif();
3     boolean checkOK=verifyA(w,c)
4     if (checkOK)
5         Output "yes "
6         return; // SUCCESS
7     return FAILURE // DO NOTHING
```

非确定性算法由两个主要过程组成：

- **Guessing**: 以非确定的方式获取一个证书 c
- **Checking**: 验证 c 是否为问题的解.
 - ⇒ 如果是, 返回 'yes'
 - ⇒ 否则, 返回 'FAILURE' (注意不是 'no')

非确定性算法由两个主要过程组成：

- **Guessing**: 以非确定的方式获取一个证书 c
- **Checking**: 验证 c 是否为问题的解.
 - ⇒ 如果是, 返回 'yes'
 - ⇒ 否则, 返回 'FAILURE' (注意不是 'no')

几点说明:

非确定性算法由两个主要过程组成：

- **Guessing**: 以非确定的方式获取一个证书 c
- **Checking**: 验证 c 是否为问题的解.
 - ⇒ 如果是, 返回 'yes'
 - ⇒ 否则, 返回 'FAILURE' (注意不是 'no')

几点说明:

- c 通常是**可行解**的一种表示形式

非确定性算法由两个主要过程组成：

- **Guessing**: 以非确定的方式获取一个证书 c
- **Checking**: 验证 c 是否为问题的解.
 - ⇒ 如果是, 返回 'yes'
 - ⇒ 否则, 返回 'FAILURE' (注意不是 'no')

几点说明:

- c 通常是**可行解**的一种表示形式
- 非确定性指的是算法使用**多个** c 来验证输入实例 w 的解

非确定性算法由两个主要过程组成：

- **Guessing**: 以非确定的方式获取一个证书 c
- **Checking**: 验证 c 是否为问题的解.
 - ⇒ 如果是, 返回 'yes'
 - ⇒ 否则, 返回 'FAILURE' (注意不是 'no')

几点说明:

- c 通常是**可行解**的一种表示形式
- 非确定性指的是算法使用**多个** c 来验证输入实例 w 的解
- 非确定性算法求解问题的**复杂性**, 包括:

非确定性算法由两个主要过程组成：

- **Guessing**: 以非确定的方式获取一个证书 c
- **Checking**: 验证 c 是否为问题的解.
 - ⇒ 如果是, 返回 'yes'
 - ⇒ 否则, 返回 'FAILURE' (注意不是 'no')

几点说明:

- c 通常是**可行解**的一种表示形式
- 非确定性指的是算法使用**多个** c 来验证输入实例 w 的解
- 非确定性算法求解问题的**复杂性**, 包括:
 - ⇒ **产生**和**验证**一个 c 的复杂性

非确定性算法由两个主要过程组成：

- **Guessing**: 以非确定的方式获取一个证书 c
- **Checking**: 验证 c 是否为问题的解.
 - ⇒ 如果是, 返回 'yes'
 - ⇒ 否则, 返回 'FAILURE' (注意不是 'no')

几点说明:

- c 通常是**可行解**的一种表示形式
- 非确定性指的是算法使用**多个** c 来验证输入实例 w 的解
- 非确定性算法求解问题的**复杂性**, 包括:
 - ⇒ **产生**和**验证**一个 c 的复杂性
 - ⇒ 所有可能的 c 的**规模**

非确定性算法由两个主要过程组成：

- **Guessing**: 以非确定的方式获取一个证书 c
- **Checking**: 验证 c 是否为问题的解。
 - ⇒ 如果是, 返回 'yes'
 - ⇒ 否则, 返回 'FAILURE' (注意不是 'no')

几点说明:

- c 通常是**可行解**的一种表示形式
- 非确定性指的是算法使用**多个** c 来验证输入实例 w 的解
- 非确定性算法求解问题的**复杂性**, 包括:
 - ⇒ **产生**和**验证**一个 c 的复杂性
 - ⇒ 所有可能的 c 的**规模**
- 注意与**算法特性**中的“**确定性**”的区别
 - ⇒ 前者指的是**获取证书**的**非确定性**
 - ⇒ 后者指的是**指令执行**的**无歧义性**

小结

掌握导致问题**易难**的可能影响因素。

- 决策问题与优化问题
- 问题实例的输入尺寸
- 非确定性算法

小结

掌握导致问题**易难**的可能影响因素。

- 决策问题与优化问题 $\Rightarrow$ **抽象问题：实例空间与解空间**
- 问题实例的输入尺寸
- 非确定性算法

小结

掌握导致问题**易难**的可能影响因素。

- 决策问题与优化问题 $\Rightarrow$ **抽象问题：实例空间与解空间**
- 问题实例的输入尺寸 $\Rightarrow$ **问题实例、实例长度、实例特征**
- 非确定性算法

小结

掌握导致问题**易难**的可能影响因素。

- 决策问题与优化问题 $\Rightarrow$ **抽象问题：实例空间与解空间**
- 问题实例的输入尺寸 $\Rightarrow$ **问题实例、实例长度、实例特征**
- 非确定性算法 $\Rightarrow$ **产生与验证、确定性算法与算法的确定性**

- ① 问题是易还是难?
- ② 什么是导致问题难解的因素?
- ③ 问题难的等价性
 - P 类问题
 - NP 类问题
 - NPC 问题

Definition (Class P)

P 类问题是能够被**确定性图灵机**在**多项式**时间内求解的决策问题集合。

Definition (Class P)

P 类问题是能够被**确定性图灵机**在**多项式**时间内求解的决策问题集合。

P 类问题可以通俗地理解为存在多项式求解算法的问题集合。

Definition (Class P)

P 类问题是能够被**确定性图灵机**在**多项式**时间内求解的决策问题集合。

P 类问题可以通俗地理解为存在多项式求解算法的问题集合。

- 多项式时间复杂性算法对于多项式的加法、乘法及复合运算是封闭的。

Definition (Class P)

P 类问题是能够被**确定性图灵机**在**多项式**时间内求解的决策问题集合。

P 类问题可以通俗地理解为存在多项式求解算法的问题集合。

- 多项式时间复杂性算法对于多项式的加法、乘法及复合运算是封闭的。
- 任何通过多项式复杂性算法的加法、乘法及复合运算能够求解的决策问题也属于类 P 问题。

Definition (Class P)

P 类问题是能够被**确定性图灵机**在**多项式**时间内求解的决策问题集合。

P 类问题可以通俗地理解为存在多项式求解算法的问题集合。

- 多项式时间复杂性算法对于多项式的加法、乘法及复合运算是封闭的。
- 任何通过多项式复杂性算法的加法、乘法及复合运算能够求解的决策问题也属于类 P 问题。
- 易解问题属于 P 类问题，如：
 - ⇒ 素数检验问题
 - ⇒ 排序问题
 - ⇒ 矩阵乘法问题等

Definition (class NP)

如果存在一个决策问题 Q 的复杂性为多项式的验证器 v ，使得对于 Q 的任意实例 w ，存在一个证书 c ，满足：

- 产生证书 c 可以在多项式时间内完成；
- 如果 c 是问题 Q 的 'yes' 答案，那么 $v(w, c) = \text{yes}$ 。

则称问题 Q 属于 NP (Non-deterministic Polynomial) 类问题。

Definition (class NP)

如果存在一个决策问题 Q 的复杂性为多项式的验证器 v ，使得对于 Q 的任意实例 w ，存在一个证书 c ，满足：

- 产生证书 c 可以在多项式时间内完成；
- 如果 c 是问题 Q 的 'yes' 答案，那么 $v(w, c) = \text{yes}$ 。

则称问题 Q 属于 NP (Non-deterministic Polynomial) 类问题。

几点说明

- 如果 w 是问题 Q 的 'no' 实例，那么对于所有有效的 c ， $v(w, c) = \text{no}$

Definition (class NP)

如果存在一个决策问题 Q 的复杂性为多项式的验证器 v ，使得对于 Q 的任意实例 w ，存在一个证书 c ，满足：

- 产生证书 c 可以在多项式时间内完成；
- 如果 c 是问题 Q 的 'yes' 答案，那么 $v(w, c) = \text{yes}$ 。

则称问题 Q 属于 NP (Non-deterministic Polynomial) 类问题。

几点说明

- 如果 w 是问题 Q 的 'no' 实例，那么对于所有有效的 c ， $v(w, c) = \text{no}$
- 如果 c 是无效的证书，即使 w 是问题 Q 的 'yes' 实例， $v(w, c) = \text{no}$

Definition (class NP)

如果存在一个决策问题 Q 的复杂性为多项式的验证器 v ，使得对于 Q 的任意实例 w ，存在一个证书 c ，满足：

- 产生证书 c 可以在多项式时间内完成；
- 如果 c 是问题 Q 的 'yes' 答案，那么 $v(w, c) = \text{yes}$ 。

则称问题 Q 属于 NP (Non-deterministic Polynomial) 类问题。

几点说明

- 如果 w 是问题 Q 的 'no' 实例，那么对于所有有效的 c ， $v(w, c) = \text{no}$
- 如果 c 是无效的证书，即使 w 是问题 Q 的 'yes' 实例， $v(w, c) = \text{no}$
- NP 类问题包含 P 类问题

Definition (class NP)

如果存在一个决策问题 Q 的复杂性为多项式的验证器 v ，使得对于 Q 的任意实例 w ，存在一个证书 c ，满足：

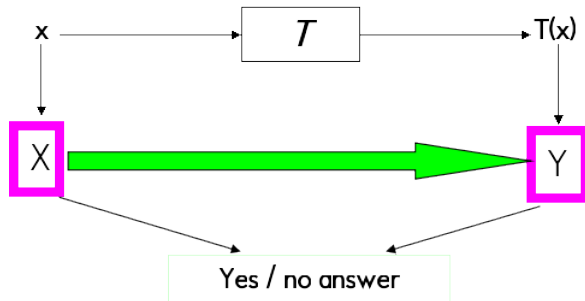
- 产生证书 c 可以在多项式时间内完成；
- 如果 c 是问题 Q 的 'yes' 答案，那么 $v(w, c) = \text{yes}$ 。

则称问题 Q 属于 NP (Non-deterministic Polynomial) 类问题。

几点说明

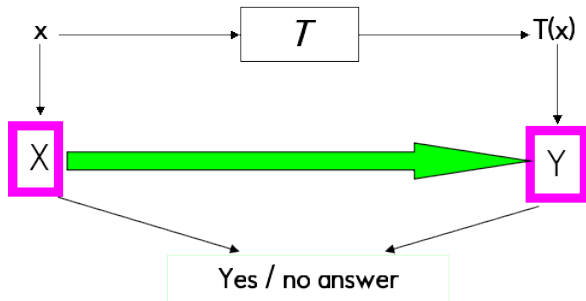
- 如果 w 是问题 Q 的 'no' 实例，那么对于所有有效的 c ， $v(w, c) = \text{no}$
- 如果 c 是无效的证书，即使 w 是问题 Q 的 'yes' 实例， $v(w, c) = \text{no}$
- NP 类问题包含 P 类问题
- NP 类问题还包括 0/1 背包、TSP 问题、子图同构问题等

多项式规约：问题 X 可以多项式地规约到问题 Y ，记做 $X \leq_P Y$ ，当且仅当存在一个多项式界的确定性算法 T ，满足：



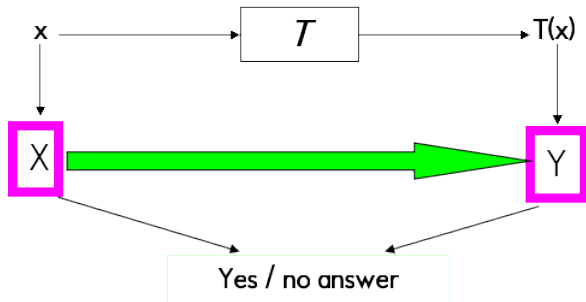
多项式规约：问题 X 可以多项式地规约到问题 Y ，记做 $X \leq_P Y$ ，当且仅当存在一个多项式界的确定性算法 T ，满足：

- 对于 X 的每个输入实例 x ， T 生成一个实例 $T(x)$



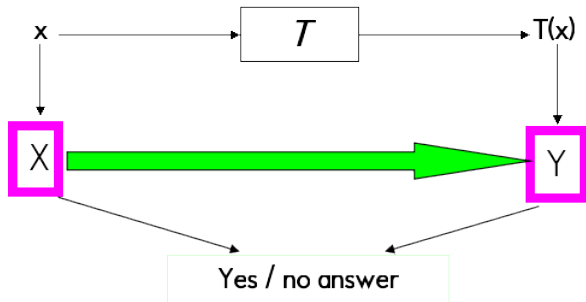
多项式规约：问题 X 可以多项式地规约到问题 Y , 记做 $X \leq_P Y$, 当且仅当存在一个多项式界的确定性算法 T , 满足：

- 对于 X 的每个输入实例 x , T 生成一个实例 $T(x)$
- x 是 X 的一个合法输入实例且对应 'yes' 答案的当且仅当 $T(x)$ 是 Y 的一个合法输入实例其对应 'yes' 答案



多项式规约：问题 X 可以多项式地规约到问题 Y ，记做 $X \leq_P Y$ ，当且仅当存在一个多项式界的确定性算法 T ，满足：

- 对于 X 的每个输入实例 x ， T 生成一个实例 $T(x)$
- x 是 X 的一个合法输入实例且对应 'yes' 答案的当且仅当 $T(x)$ 是 Y 的一个合法输入实例其对应 'yes' 答案



$X \leq_P Y$ 意味着 X 至多与 Y 一样难。

Theorem 1

如果 $X \leq_P Y$ 并且 Y 属于 P 类问题, 那么 X 也属于 P 类问题。

Proof.

Theorem 2

如果 $X \leq_P Y$ 并且 Y 属于 P 类问题, 那么 X 也属于 P 类问题。

Proof.

假设 x 是 X 的任意一个实例, 因为 $X \leq_P Y$, 我们可以通过求解 Y 来求解 X 。

Theorem 3

如果 $X \leq_P Y$ 并且 Y 属于 P 类问题, 那么 X 也属于 P 类问题。

Proof.

假设 x 是 X 的任意一个实例, 因为 $X \leq_P Y$, 我们可以通过求解 Y 来求解 X 。

- 首先将 x 变换为 $T(x)$, 复杂性为 $O(p(|x|))$, 其中 p 为多项式函数

Theorem 4

如果 $X \leq_P Y$ 并且 Y 属于 P 类问题, 那么 X 也属于 P 类问题。

Proof.

假设 x 是 X 的任意一个实例, 因为 $X \leq_P Y$, 我们可以通过求解 Y 来求解 X 。

- 首先将 x 变换为 $T(x)$, 复杂性为 $O(p(|x|))$, 其中 p 为多项式函数
- 由于 Y 属于 P 类问题, 所以存在复杂性为多项式界 q 的算法求解 Y

Theorem 5

如果 $X \leq_P Y$ 并且 Y 属于 P 类问题, 那么 X 也属于 P 类问题。

Proof.

假设 x 是 X 的任意一个实例, 因为 $X \leq_P Y$, 我们可以通过求解 Y 来求解 X 。

- 首先将 x 变换为 $T(x)$, 复杂性为 $O(p(|x|))$, 其中 p 为多项式函数
- 由于 Y 属于 P 类问题, 所以存在复杂性为多项式界 q 的算法求解 Y
- 将 $T(x)$ 作为 Y 的输入进行求解, 其复杂性为 $O(q(p(|x|)))$

Theorem 6

如果 $X \leq_P Y$ 并且 Y 属于 P 类问题, 那么 X 也属于 P 类问题。

Proof.

假设 x 是 X 的任意一个实例, 因为 $X \leq_P Y$, 我们可以通过求解 Y 来求解 X 。

- 首先将 x 变换为 $T(x)$, 复杂性为 $O(p(|x|))$, 其中 p 为多项式函数
- 由于 Y 属于 P 类问题, 所以存在复杂性为多项式界 q 的算法求解 Y
- 将 $T(x)$ 作为 Y 的输入进行求解, 其复杂性为 $O(q(p(|x|)))$
- 因而求解 x 的复杂性为 $O(p(x) + q(p(|x|)))$

Theorem 7

如果 $X \leq_P Y$ 并且 Y 属于 P 类问题, 那么 X 也属于 P 类问题。

Proof.

假设 x 是 X 的任意一个实例, 因为 $X \leq_P Y$, 我们可以通过求解 Y 来求解 X 。

- 首先将 x 变换为 $T(x)$, 复杂性为 $O(p(|x|))$, 其中 p 为多项式函数
- 由于 Y 属于 P 类问题, 所以存在复杂性为多项式界 q 的算法求解 Y
- 将 $T(x)$ 作为 Y 的输入进行求解, 其复杂性为 $O(q(p(|x|)))$
- 因而求解 x 的复杂性为 $O(p(x) + q(p(|x|)))$

因而 X 也属于 P 类问题



Definition

如果所有的NP类问题都可以多项式地规约到问题 Q ，那么称 Q 为NP-hard问题。

Definition

如果所有的NP类问题都可以多项式地规约到问题 Q，那么称 Q 为NP-hard问题。

Definition

如果问题 Q 是NP-hard问题并且也NP类问题，称 Q 为 NP Complete(NPC) 问题。

Definition

如果所有的NP类问题都可以多项式地规约到问题 Q，那么称 Q 为NP-hard问题。

Definition

如果问题 Q 是NP-hard问题并且也NP类问题，称 Q 为 NP Complete(NPC) 问题。

几点说明：

Definition

如果所有的NP类问题都可以多项式地规约到问题 Q，那么称 Q 为NP-hard问题。

Definition

如果问题 Q 是NP-hard问题并且也NP类问题，称 Q 为 NP Complete(NPC) 问题。

几点说明：

- 所有的 NPC 问题关于多项式规约是封闭的（自反的、对称的和传递的）

Definition

如果所有的NP类问题都可以多项式地规约到问题 Q，那么称 Q 为NP-hard问题。

Definition

如果问题 Q 是NP-hard问题并且也NP类问题，称 Q 为 NP Complete(NPC) 问题。

几点说明：

- 所有的 NPC 问题关于多项式规约是封闭的（自反的、对称的和传递的）
- 如果能够找到一个 NPC 问题的多项式算法，那么 $P = NP$

Definition

如果所有的NP类问题都可以多项式地规约到问题 Q ，那么称 Q 为NP-hard问题。

Definition

如果问题 Q 是NP-hard问题并且也NP类问题，称 Q 为 NP Complete(NPC) 问题。

几点说明：

- 所有的 NPC 问题关于多项式规约是封闭的（自反的、对称的和传递的）
- 如果能够找到一个 NPC 问题的多项式算法，那么 $P = NP$
- $P = NP$ 问题位于七个“千禧年数学难题”之首，迄今无解

典型 NPC 问题列表

- Boolean satisfiability problem (3-SAT)
- N-puzzle
- Knapsack problem
- Hamiltonian path problem
- Traveling salesman problem
- Subgraph isomorphism problem
- Subset sum problem
- Clique problem
- Vertex cover problem
- Independent set problem
- Graph coloring problem

K^{th} largest subset problem

Instance:

- A finite set A of positive integers,
- Two nonnegative integers $K \leq 2^{|A|}$ and $B \leq \sum_{a \in A} a$.

Question: Are there at least K distinct subsets $Y \subseteq A$ such that

$$\sum_{a \in Y} a \leq B \quad (1)$$

or

K^{th} largest subset problem

Instance:

- A finite set A of positive integers,
- Two nonnegative integers $K \leq 2^{|A|}$ and $B \leq \sum_{a \in A} a$.

Question: Are there at least K distinct subsets $Y \subseteq A$ such that

$$\sum_{a \in Y} a \leq B \quad (1)$$

or

$$\left\| Y \subseteq A : \sum_{a \in Y} a \leq B \right\| \geq K \quad (2)$$

K^{th} largest subset problem

Instance:

- A finite set A of positive integers,
- Two nonnegative integers $K \leq 2^{|A|}$ and $B \leq \sum_{a \in A} a$.

Question: Are there at least K distinct subsets $Y \subseteq A$ such that

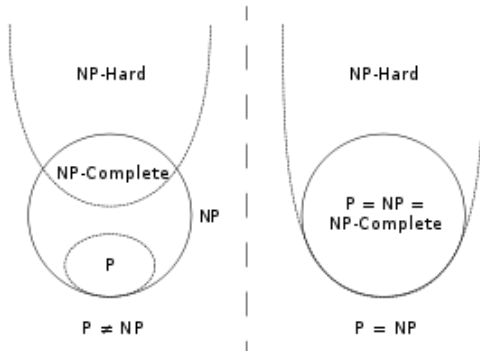
$$\sum_{a \in Y} a \leq B \quad (1)$$

or

$$\left\| Y \subseteq A : \sum_{a \in Y} a \leq B \right\| \geq K \quad (2)$$

我们容易证明这个问题是 NP-hard 问题，但无法证明是否属于 NP 类问题

Euler diagram



小结

N-

NondeterministicAlgorithms

- Deterministic algorithm: Given a particular input, it will always produce the same correct output
- Non-deterministic algorithm: with one or more choice points where multiple different continuations are possible, without any specification of which one will be taken

小结

N- **Nondeterministic** Algorithms

- Deterministic algorithm: Given a particular input, it will always produce the same correct output
- Non-deterministic algorithm: with one or more choice points where multiple different continuations are possible, without any specification of which one will be taken

P- **Polynomial** Time complexity

- Computable
- Polynomial time is assumed the lowest complexity

小结

- N- **Nondeterministic** Algorithms
- Deterministic algorithm: Given a particular input, it will always produce the same correct output
 - Non-deterministic algorithm: with one or more choice points where multiple different continuations are possible, without any specification of which one will be taken
- P- **Polynomial** Time complexity
- Computable
 - Polynomial time is assumed the lowest complexity
- C- **Complete** transform closed
- Reducible

Coming up: The End.

Thank You